

Smart Fridge

IoT Dashboard

Low-Cost Smart Refrigerator

with Predictive Spoilage Alerts

A Full-Stack MERN + IoT Research Project

TEAM MEMBERS

Aatiqah Harmine
2303A51860

Anushka Voorugonda
2303A51176

Neha Fariyal
2303A51598

B. Spoorthi
2303A51802

Introduction -The Problem of Food Waste

13.2%

of food lost
before consumers

19%

lost in households
worldwide

1/3

of all food produced
is wasted globally



The Problem

- Traditional refrigerators do not track food items
- No real-time monitoring of temperature or freshness
- Leads to food waste and health issues



Our Solution

- ✓ A low-cost Smart Refrigerator System using IoT sensors and AI
- ✓ Monitors food condition in real-time
- ✓ Provides alerts before spoilage

Project Objectives

01

Real-Time Monitoring

Monitor temperature, humidity, and food quantity inside the fridge in real-time from ESP32 hardware to a web dashboard via REST API.

02

Predictive Spoilage Alerts

Integrate Gemini AI Use AI to predict when food may spoil and alert the user early.

03

Full-Stack Web Application

Deliver a responsive MERN-stack dashboard with JWT authentication, CRUD food inventory management, and live chart visualizations.

04

IoT Integration

Connect sensors and ESP32 to send data to the cloud system.

05

Low-Cost & Accessible Design

Design an affordable system that can work with a normal fridge.

06

Scalable Architecture

Create a system that can support multiple users and devices.

Stakeholders



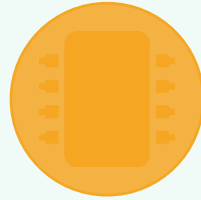
End Users (Households)

- ☐ Users can check food freshness easily
- ☐ They get alerts before food spoils
- ☐ Helps reduce food waste



System Administrators

- ☐ Manage users and system settings
- ☐ Monitor system performance
- ☐ Maintain the backend



IoT / Hardware Engineers

- ☐ Set up sensors and ESP32
- ☐ Ensure data is collected properly
- ☐ Connect hardware to the system



AI & Data Analysts

- ☐ Analyze sensor data
- ☐ Improve spoilage predictions
- ☐ Make the system smarter

Programming Languages, Tools & Software

Hardware (IoT Layer)

- ESP32 MCU · Wi-Fi Dual-core
- DHT22 · Temp & Humidity
- HX711 + Load Cell · Weight
- HC-SR501 · PIR Motion
- HC-SR04 · Ultrasonic x2

Communication

- MQTT Protocol
- EMQX Broker (primary)
- HiveMQ (fallback)
- Wi-Fi 802.11 b/g/n
- REST API (HTTP)

Backend

- Node.js + Express.js
- MongoDB Atlas (NoSQL)
- Mongoose ODM
- JWT Authentication
- Node-RED IoT Platform

Frontend

- React 18 + Vite
- Chart.js + react-chartjs-2
- React Router v6
- CSS Modules
- Axios HTTP client

AI & Analytics

- Gemini AI API
- IoT Deployment Level 4
- Predictive Spoilage Model
- Real-time Analytics
- Explainable AI (XAI)

Dev Tools

- VS Code
- Arduino IDE
- Figma (UI design)
- GitHub (version control)
- Thunder Client (API test)

Prompts Used to Generate Code (Part 1 of 2)

P1

Project Scaffold

"give me dashboard for this application/research paper using react js ,mongo db."

P2

Sensor Data Model

"Create a Mongoose schema for IoT sensor readings with fields: temperature (°C), humidity (%), weightTotal (kg), motionDetected (bool), distanceTop and distanceBottom (cm), timestamps, and an auto-generated spoilageRisk field."

P3

Dashboard Page

"give login page and dash board generate mongo db ,node js/react.js and give me code how to implement in vscode. "

P4

JWT Auth Middleware

"Write Express.js middleware that validates a Bearer JWT token from the Authorization header, decodes the payload, attaches req.user, and returns 401 on failure."

Prompts Used to Generate Code (Part 2 of 2)

P5

Food Inventory CRUD

"Create Express.js routes for food inventory: GET all active items, POST new item with name/category/quantity/expiryDate, PUT update item, DELETE (soft-delete), and GET items expiring within N days."

P6

Gemini AI Spoilage Analysis

"Integrate Google Gemini AI API into the Node.js backend. Pass current temperature, humidity, and food inventory data as context and prompt it to return a JSON spoilage risk assessment with recommendations."

P7

Alerts System

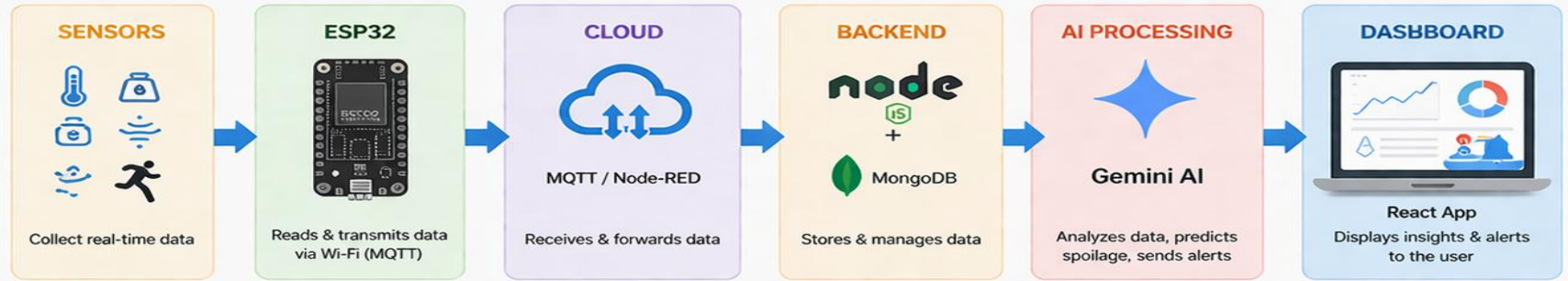
"Build an Express alerts route and Mongoose model for IoT and AI-generated alerts. Include severity (info/warning/critical), isRead flag, GET all alerts, PUT mark-as-read (single and all), and auto-create alerts when sensor thresholds are exceeded."

P8

React Auth Context

"Create a React Context and Provider for JWT authentication. Include Login(), Logout(), register() functions using Axios, persist the token in localStorage, expose user state, and protect routes with a PrivateRoute component."

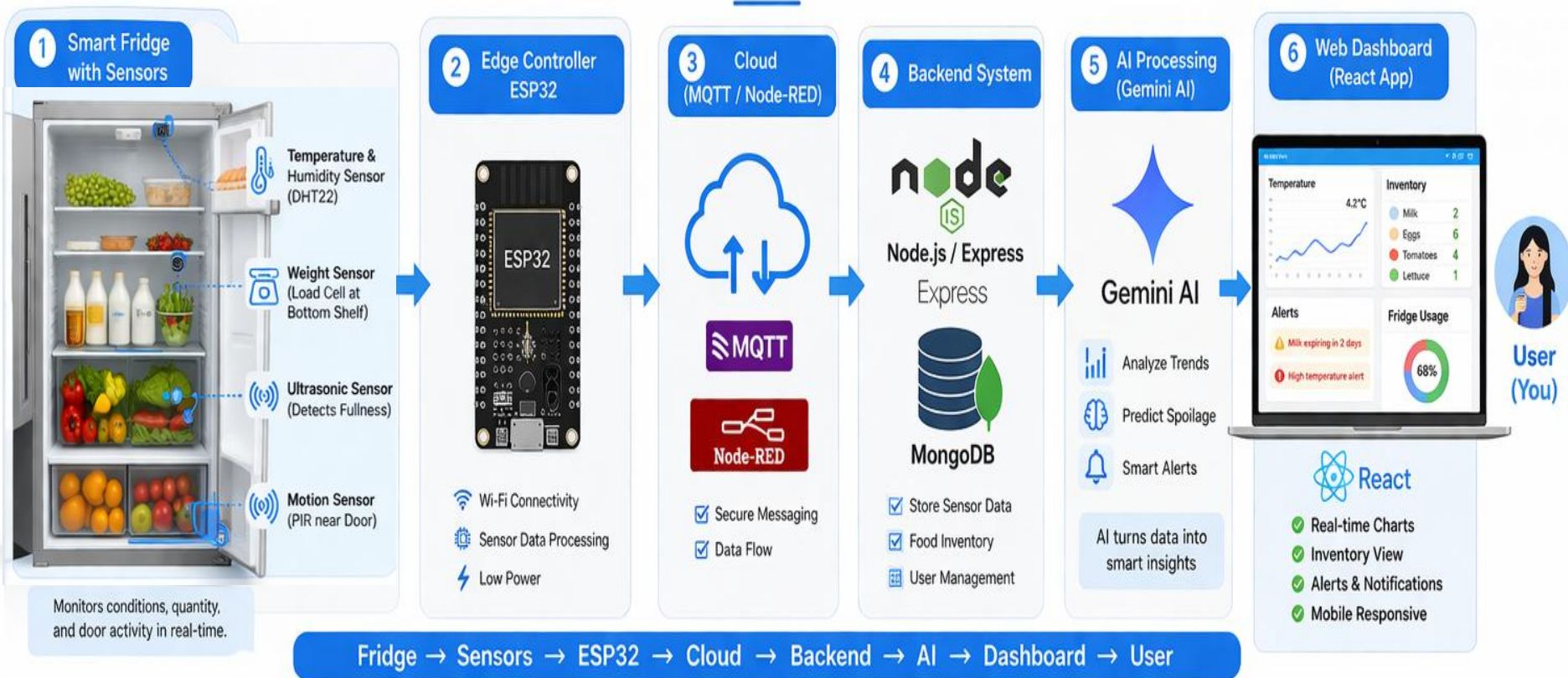
System Workflow



How it Works

- Sensors collect data (temperature, weight)
- ESP32 sends data to cloud via MQTT
- Backend API stores data in MongoDB
- AI analyzes data and predicts spoilage
- Dashboard displays results

System Workflow



SYSTEM DESIGN

Smart Fridge IoT Architecture



Temperature & Humidity Sensor (DHT22)

Monitors temperature and humidity.



Ultrasonic Sensor (HC-SR04)

Measures shelf space / food level.



Weight Sensor (Load Cell)

Measures the weight of food items.



Motion Sensor (PIR)

Detects door opening/closing.



Better Monitoring

Real-time visibility of fridge conditions.



Less Waste

AI predicts spoilage and helps you act early.



Energy Efficient

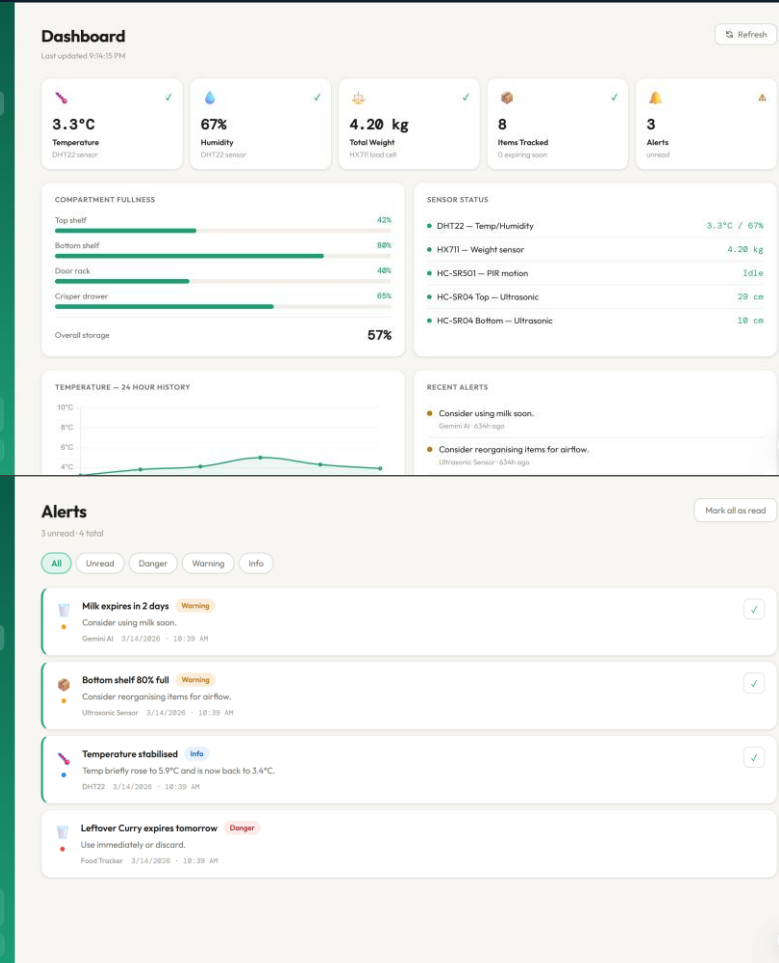
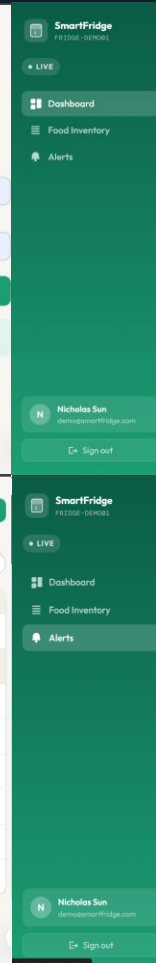
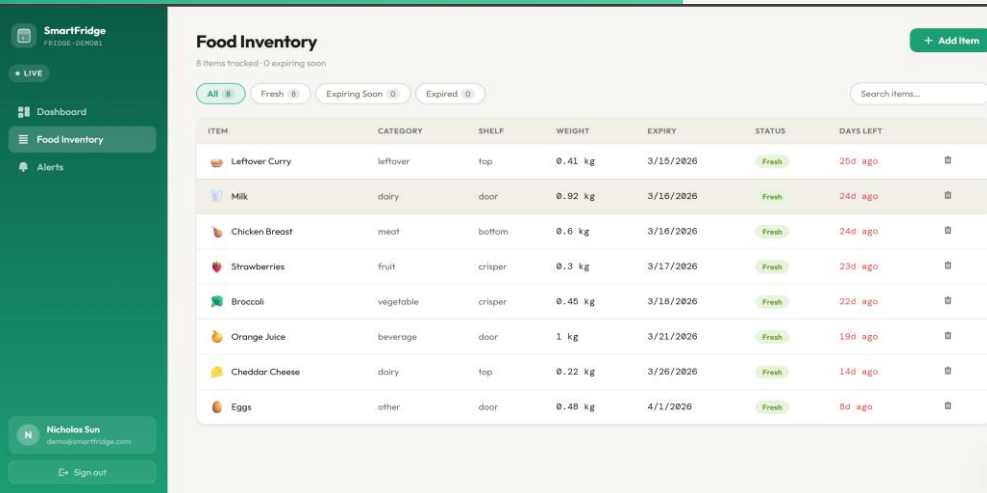
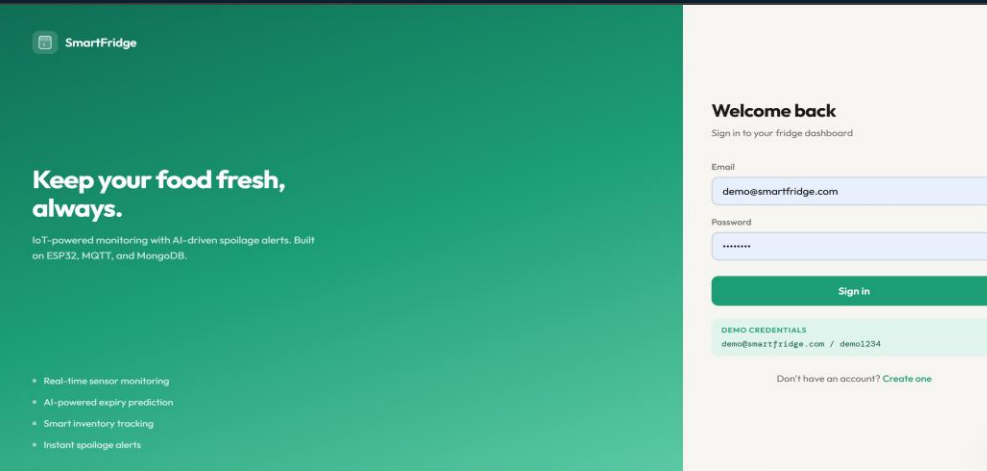
Motion sensor reduces unnecessary cooling.



Smarter Decisions

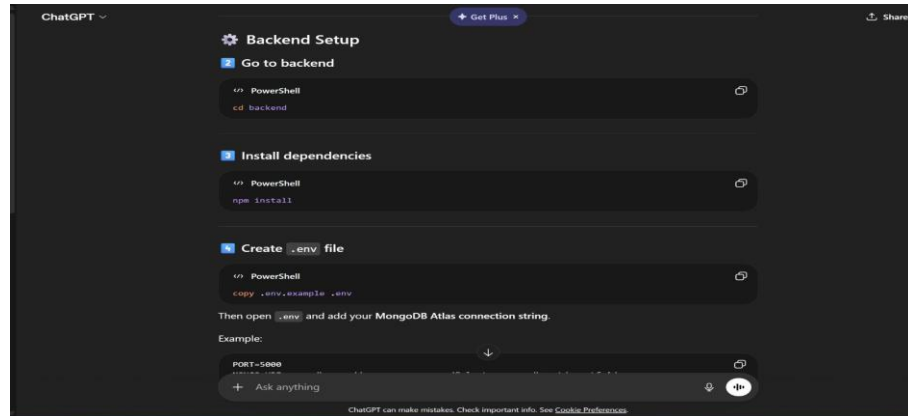
Data-driven insights for a healthier lifestyle.

Front-End Application Views

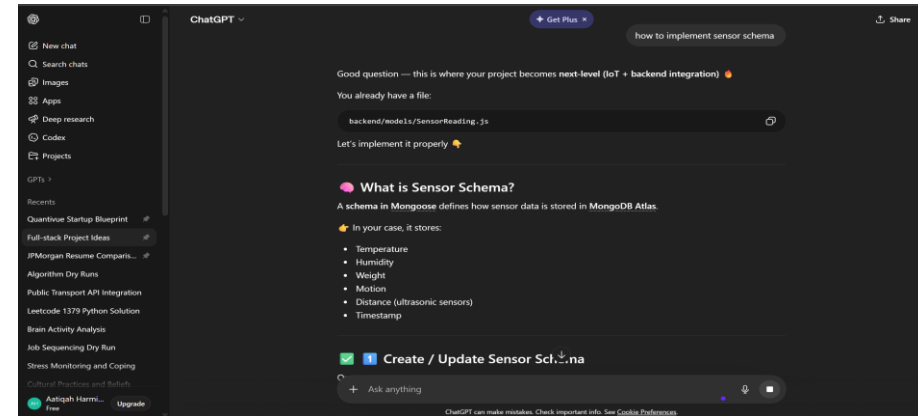


Proof of Prompts Generating Code

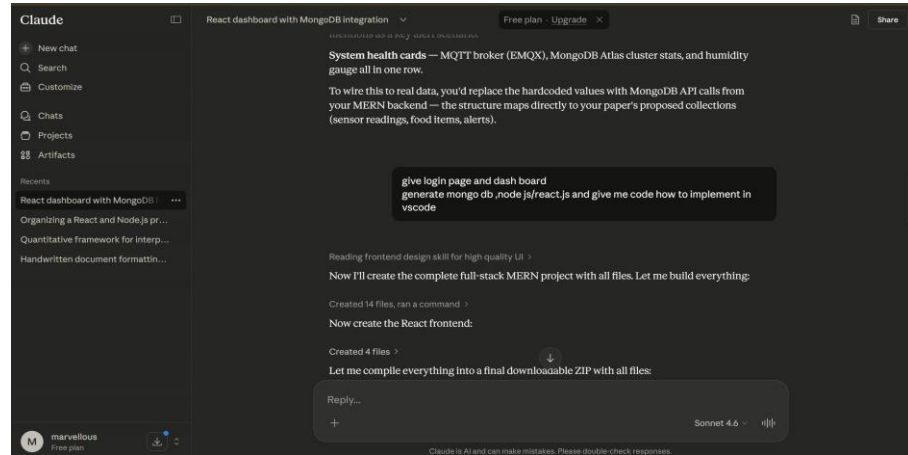
Screenshot 1 — Backend Server (P1)



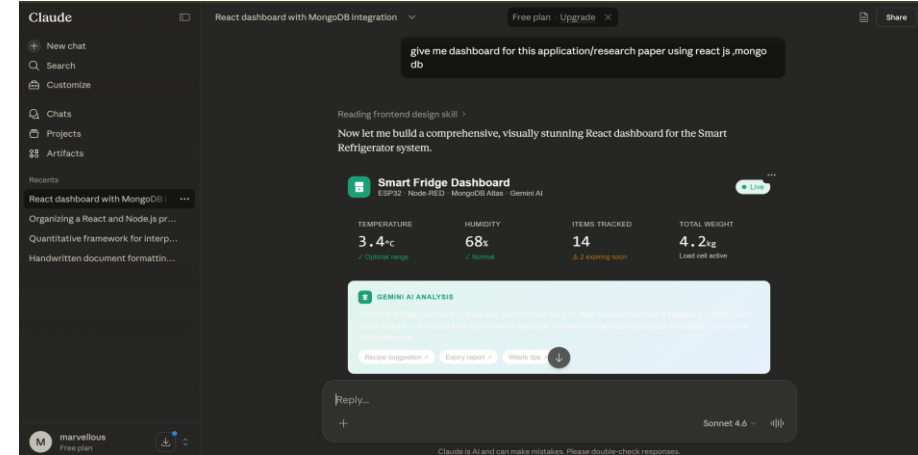
Screenshot 2 — Sensor Schema (P2)



Screenshot 3 — Dashboard React Page (P3)



Screenshot 4 — Gemini AI Integration (P6)



References

[1] Recommendations for Developing a Low-Cost Smart Refrigerator with Predictive Spoilage Alerts. <https://ieeexplore.ieee.org/abstract/document/11158709>

[2] Meta Open Source. (2024). React Documentation. <https://react.dev/>

[3] OpenJS Foundation. (2024). Node.js Documentation. <https://nodejs.org/en/docs/>

[4] Expressjs.com. (2024). Express.js Guide. <https://expressjs.com/>

[5] Google AI. (2024). Gemini API Documentation. <https://ai.google.dev/>

[6] EMQX. (2024). EMQX MQTT Broker Documentation. <https://www.emqx.io/docs/>

[7] Node-RED. (2024). Node-RED Documentation. <https://nodered.org/docs/>

[8] Espressif Systems. (2024). ESP32 Technical Reference. <https://docs.espressif.com/>

[9] npm, Inc. (2024). PptxGenJS Library. <https://gitbrent.github.io/PptxGenJS/>

[10] Vitejs.dev. (2024). Vite Build Tool Documentation. <https://vitejs.dev/guide/>

AI Tools & How They Were Used

Claude (Anthropic)

Primary Code Generator

- Full MERN stack backend (Node.js, Express, MongoDB schemas)
- React frontend: Login, Dashboard, Food Inventory, Alerts pages
- JWT authentication middleware and route protection
- Database seed scripts and REST API endpoints
- This PowerPoint presentation generated automatically

Gemini AI (Google)

IoT Analytics Engine

- Real-time spoilage prediction from sensor data
- Recipe suggestions based on expiring ingredients
- Food freshness analysis from temperature/humidity patterns
- Integrated via Gemini AI API within the dashboard
- Explainable AI alerts sent to user dashboard

ChatGPT (OpenAI)

Research & Planning

- Literature review assistance and research synthesis
- System architecture planning and IoT topology design
- Agile sprint planning and project timeline structuring
- Arduino/ESP32 code review and sensor integration advice
- Debugging assistance for MQTT broker configuration

All Prompts Used to Complete This Project

Category	Prompt Description
Backend	Create a MongoDB schema for a smart refrigerator with User, Sensor Reading, FoodItem, and Alert collections.
Backend	Build a Node.js + Express REST API with JWT auth, sensor routes, food CRUD, and dashboard summary endpoint.
Backend	Write a MongoDB Atlas seed script with demo user, 24h sensor data, 8 food items, and sample alerts.
Frontend	Design a React split-layout login page with teal gradient panel, form validation, and JWT integration.
Frontend	Build a full React dashboard fetching /api/dashboard/summary with metric cards, Chart.js temp history, and food grid.
Frontend	Create a React food inventory page with modal add form, filter tabs, expiry status pills, and delete API calls.
Frontend	Build a React alerts page with severity color coding, mark-as-read, filter tabs, and real-time refresh.
Frontend	Create a React Router + Context API app structure with protected routes and persistent JWT token storage.
IoT	Write an Arduino sketch for ESP32 to read DHT22, HX711, HC-SR501 PIR, and two HC-SR04 sensors, then POST to REST API.
Deployment	Write a VS Code step-by-step setup guide: MongoDB Atlas config, backend .env, npm install, seed, and concurrent dev servers.
AI	Explain how to integrate the Gemini AI API into a MERN app for real-time food spoilage prediction and recipe suggestions.